

How the PMS QA Automation Works

A plain-language walkthrough for someone seeing this project for the first time

You do **not** need prior Playwright or testing experience to read this. It explains what the project is, why it exists, how it is put together, and how to run it. Technical terms are defined the first time they appear, and there is a glossary at the end.

1. What is this project, in one paragraph?

pms-qa-automation is a separate project whose only job is to **automatically test the PMS application and report bugs**. It logs into the real apps and calls the real backend services the way a user or the frontend would, checks that they behave correctly, and writes up anything that looks wrong. It never changes the application's code and never fixes bugs on its own — it is an independent inspector, not a mechanic.

Analogy. Think of the PMS platform as a building. This project is the **building inspector** who walks through every room, tries every lock, and files a report on what's broken. The inspector doesn't repair anything — they hand the report to the builders.

2. What does "PMS" look like, so the tests make sense?

The thing being tested has three parts:

Part	What it is	Address (on staging)
Admin frontend	The web app staff use	<code>/admin</code>
Client frontend	The web app customers use	<code>/client</code>
15 backend services	Small programs that store and serve data (auth, jobs, invoices, contacts, ...)	one shared web address, different path per service

Two ideas matter for the rest of this guide:

- **Multi-tenant:** many separate customer organisations ("tenants") share the same system, and each must **only ever see its own data**. A huge part of testing is proving Tenant A can't see or touch Tenant B's data.
- **Token-based login:** after you log in, the server gives your browser a signed pass (a "token"). Every later request carries that token so the server knows who you are and which tenant you belong to.

3. What kinds of things does the automation check?

Test type	Question it answers
Smoke tests	"Is everything switched on?" Do the apps load, do the login screens appear, does each of the 15 services respond?

Auth-boundary tests	"Are the doors locked?" Does every service reject a request that has no valid login token?
Tenant-isolation tests	"Can one customer peek at another's data?" Logged in as Tenant A, try to read/change Tenant B's records — and expect to be refused.
Critical-flow tests	"Do the important journeys work end to end?" e.g. admin logs in, creates a job, and the job shows up.

4. The tools it uses

- **Playwright** — the testing tool. It can both drive a real web browser (click, type, read the screen) *and* call backend services directly. Using one tool for both keeps everything in a single report. It's the only major dependency.
- **TypeScript** — the language the tests are written in (JavaScript with type checking, which catches mistakes before the tests even run).

5. A tour of the project folders

Everything lives under `pms-qa-automation/`. Here is what each folder is for:

Folder / file	What's inside
<code>tests/</code>	The actual tests, grouped by kind: <code>smoke/</code> , <code>admin/</code> , <code>client/</code> , <code>api/</code> (with <code>auth/</code> and <code>tenancy/</code>), and <code>integration/</code> (the critical flows).
<code>pages/</code>	Page Objects — one file per screen (e.g. the admin login page). They hold the "how to find the email box and click Login" details, so if a screen changes you fix it in one place. Tests read cleanly: <code>loginPage.loginAs(email, password)</code> .
<code>fixtures/</code>	Reusable setup handed to tests automatically — e.g. "give me a browser that is already logged in as the admin," or "give me Tenant A and Tenant B identities."
<code>helpers/</code>	Shared toolbox: the API client, environment config, the service list, result assertions, and the bug-report writer.
<code>scripts/</code>	Run-once helpers: global setup/teardown, and the HTML→PDF report generators.
<code>bug-reports/</code>	The output that matters — one Markdown file per confirmed bug, kept as a durable record.
<code>playwright.config.ts</code>	The master settings file: which groups of tests exist, timeouts, and what evidence to capture on failure.
<code>.env.staging</code>	The addresses and QA login details for the staging environment. Kept out of version control because it holds passwords.
<code>ARCHITECTURE.md</code> / <code>CHARTER.md</code> / <code>README.md</code>	Reference docs: how the app is built, the rules QA must follow, and how to run things.

6. What happens, step by step, when you run the tests

- 1 **Config loads.** `playwright.config.ts` reads which environment you asked for (via `TEST_ENV`) and loads the matching `.env` file — the URLs and QA credentials.
- 2 **Global setup runs once.** It confirms the QA logins actually work, so if a password is wrong you get one clear error instead of every test failing mysteriously.
- 3 **Each test gets its fixtures.** If a test asks for an "admin browser," the fixture logs in first and hands over a ready-to-use, authenticated page. The test doesn't repeat login code.
- 4 **The test does its checks.** It clicks/types in the browser, or calls a service, then *asserts* — states what must be true (e.g. "this must return 401"). If reality differs, the test fails.
- 5 **Evidence is captured on failure.** A screenshot, a video, and a "trace" (a step-by-step replay incl. network calls) are saved automatically — no guesswork about what went wrong.

- 6 **A bug report is written.** Security tests write a precise, pre-classified report; any other failure gets an automatic "needs investigation" skeleton. Nothing fails silently.
- 7 **Cleanup + summary.** Any test data the run created is removed, and you get a pass/fail summary plus an HTML report you can open in a browser.

7. How login works inside the tests

Because almost everything needs a logged-in user, the project handles login in reusable **fixtures** so individual tests stay short:

- **API sessions** — for backend tests, the fixture calls the login endpoint once and keeps the token. A test then says `apis.jobs.as(adminSession).get('/...')` — "call the jobs service as this admin."
- **Browser sessions** — for UI tests, the fixture opens the login page, fills it in, and hands back a browser that's already inside the app.
- **Two tenants** — a special fixture provides Tenant A and Tenant B side by side, which is exactly what isolation tests need: do something as A, prove B is unaffected (and vice-versa).

Why two tenants? You cannot prove "customers are isolated from each other" with only one customer. Isolation tests always need a second, independent tenant to attempt (and be denied) access to.

8. What a real test looks like (annotated)

This is the shape of an auth-boundary test — readable even if you've never seen the codebase:

```
test('jobs-backend rejects an unauthenticated request', async ({ request }) => {
  // Call the jobs service WITHOUT a login token:
  const response = await apiClients(request).jobs.get('/workflow/jobs/jobs');

  // State what MUST be true. 401 means "Unauthorized" — the door is locked.
  expect(response.status()).toBe(401);
});
```

If the service wrongly answered `200` (success) to a request with no token, this line fails, and the report flags an open door. That is literally how three real security gaps in this project were caught.

9. Environments and the safety rules

The same tests can point at different environments, chosen by the `TEST_ENV` setting:

Environment	Meaning	Safety
local	Services running on your own machine	Anything goes
staging	A test copy of the live system	Safe to create/modify QA data
production	The real, customer-facing system	Hard-blocked. Tests refuse to run against production unless someone very deliberately overrides it, and even then only read-only smoke checks are allowed.

Golden rules the project enforces. QA never edits application code to "fix" a bug — it reports it. Any data the tests create is clearly QA-labelled, tracked, and cleaned up. Production is never modified. These aren't just guidelines; they're built into the framework (e.g. destructive helpers throw an error if pointed at production).

10. How to run it (copy-paste)

Open a terminal in the project folder first:

```
cd "C:/PMS Multitenant/pms-qa-automation"
```

First time only — install the tool and a browser:

```
npm install  
npm run install:browsers
```

Run the whole suite against staging:

```
npm run test:staging
```

Run just one group (faster while you're focused):

```
npm run test:staging -- --project=smoke-api      # backend health only  
npm run test:staging -- --grep-invert "isolation" # everything except cross-tenant tests
```

See the visual report after a run:

```
npm run report
```

11. How to read the results

- **Passed** — the check held; the behaviour is correct.
- **Failed** — reality didn't match what the test required. A bug report and evidence are saved.
- **Skipped / Blocked** — the test couldn't run meaningfully yet (e.g. missing test data, or a step deliberately gated for safety). This is *not* the same as a failure.

Failures point you to `bug-reports/` (the written findings) and `test-results/` (screenshots, videos, traces). A run that shows failures is often the automation doing its job — the current staging run is green apart from the genuine open bugs it is meant to keep flagging.

12. Glossary

Term	Plain meaning
Playwright	The testing tool that drives browsers and calls services.
Test / spec	A single automated check, written in a <code>.spec.ts</code> file.
Assertion (<code>expect</code>)	A statement of what must be true; if it isn't, the test fails.
Fixture	Reusable setup automatically provided to a test (e.g. a logged-in browser).
Page Object	A file that hides the details of one screen so tests read simply.
Token	The signed pass proving who you are on each request after login.
Tenant	One customer organisation; its data must stay private from other tenants.
Endpoint	A specific address on a backend service, e.g. <code>/api/auth/login</code> .
Status code	The server's short answer: 200 = OK, 401 = not logged in, 403/404 = not allowed / not found, 500 = server error.
Smoke test	A quick "is it switched on and responding?" check.
Trace	A recorded step-by-step replay of a test, incl. network activity, for debugging.
Environment	Which copy of the system you're testing: local, staging, or production.

13. Where to go next

- `README.md` — the practical run guide and command reference.
- `ARCHITECTURE.md` — how the PMS system is built (routes, login, tenancy), the ground truth the tests rely on.
- `CHARTER.md` — the rules: what QA may and may not do.
- `bug-reports/` — the findings so far, each with reproduction steps and evidence.

PMS QA Automation — beginner's guide. Generated for onboarding; pair it with the README for hands-on commands.